

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
SOFTWARE ENGINEERING – LECTURE 03

Requirements Engineering: Elicitation And Analysis

Prof. Luigi Libero Lucio Starace

luigiliberolucio.starace@unina.it

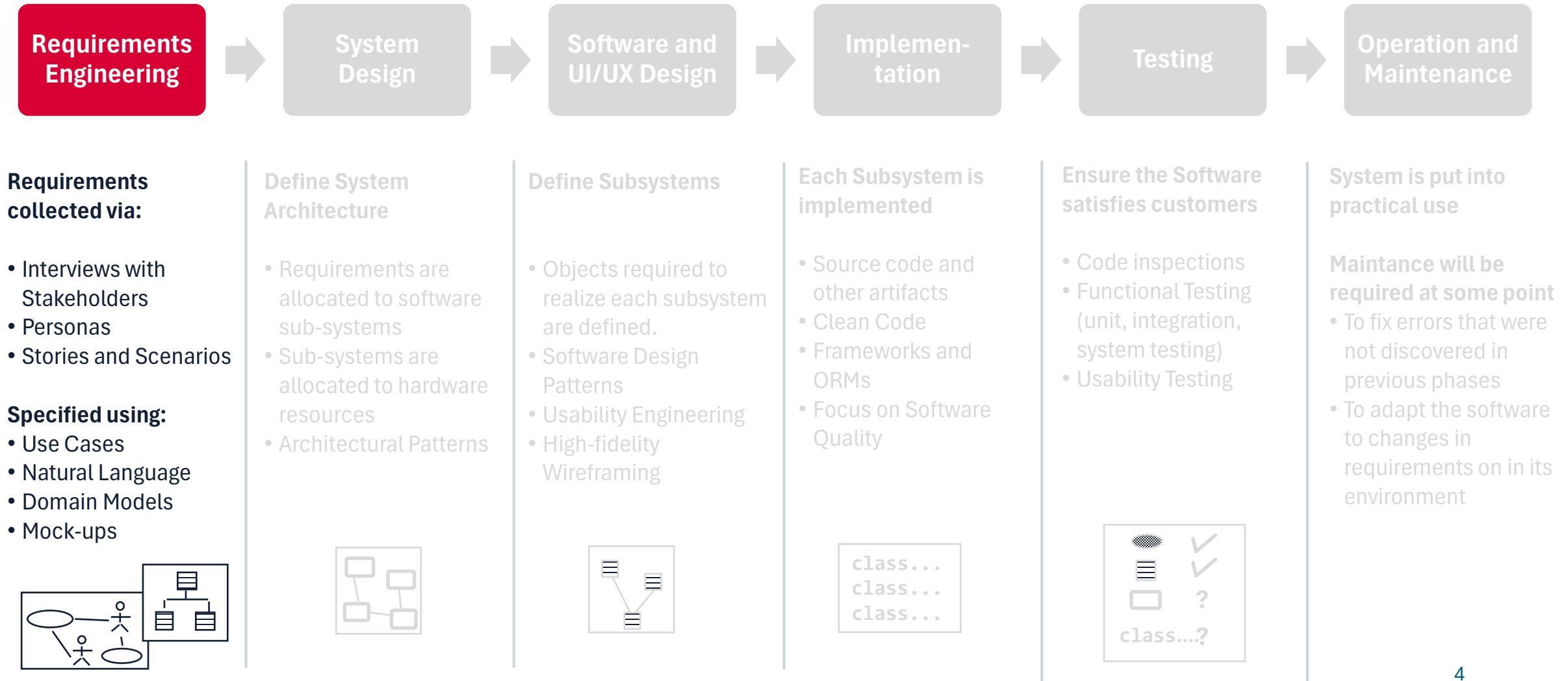
<https://luistar.github.io>

<https://www.docenti.unina.it/luigiliberolucio.starace>

Previously, on Software Engineering

- We've discussed what Software Engineering is and why it's important
- We've talked about **Software Processes** and **SDLC**
 - We had an overview of the **Waterfall** Software Process Model
- We've seen the many aspects of **Software Quality**
- Today, we'll get into Requirements Engineering

The Software Life Cycle



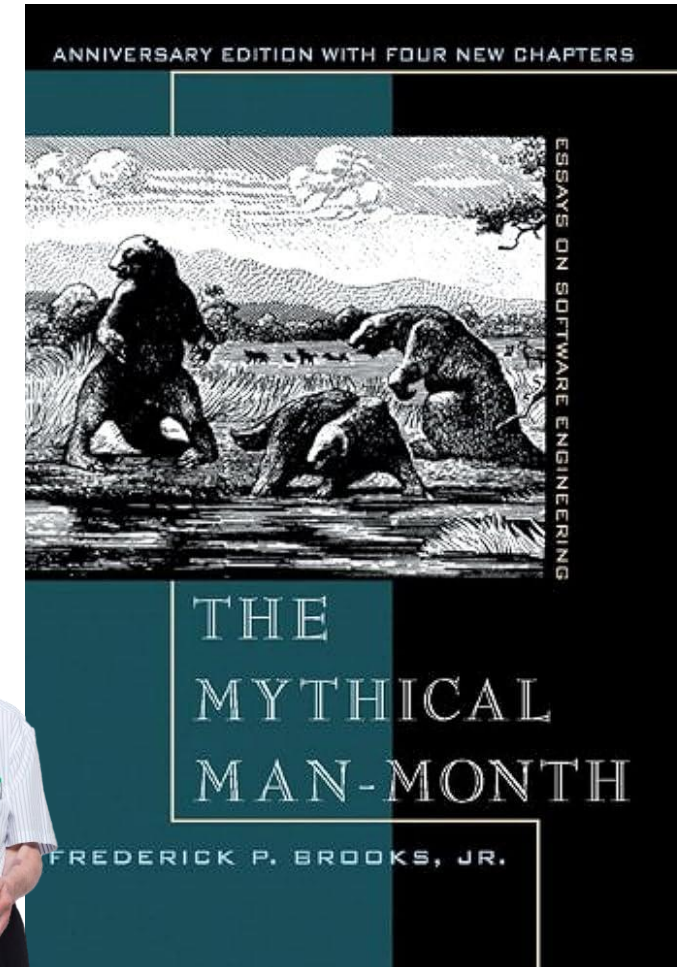
Fred Brooks on Requirements Engineering

*The hardest single part of building a software system is deciding precisely **what to build**.*

No other part of the conceptual work is so difficult as establishing the detailed technical requirements [...].

No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.

- Fred P. Brooks



Software Requirements

- Requirements are descriptions of what the system should do:
 - **services** that the system should provide to its users
 - **operational constraints**
- Some examples:
 - The system shall allow students to search for courses based on keywords
 - The system shall allow students to enroll in courses
 - Users interfaces shall be implemented as native Android and iOS apps
 - The system shall handle at least 10k concurrent students
 - The system shall authenticate users also by using built-in fingerprint sensors

Requirements: Different Levels

The term «**requirement**» is used inconsistently in the Software Industry

- In some cases, a requirement is an abstract, high-level description of a service the system should provide or a constraint on the system.
 - These are called **User Requirements** (focus is on end users' perspective)
- In others, it is a more detailed and formal definition of a system function
 - These are called **System Requirements** (focus is on the system to be built)

Requirements: Different Levels

User Requirement

1. The system should generate a monthly course enrollment report



System Requirements

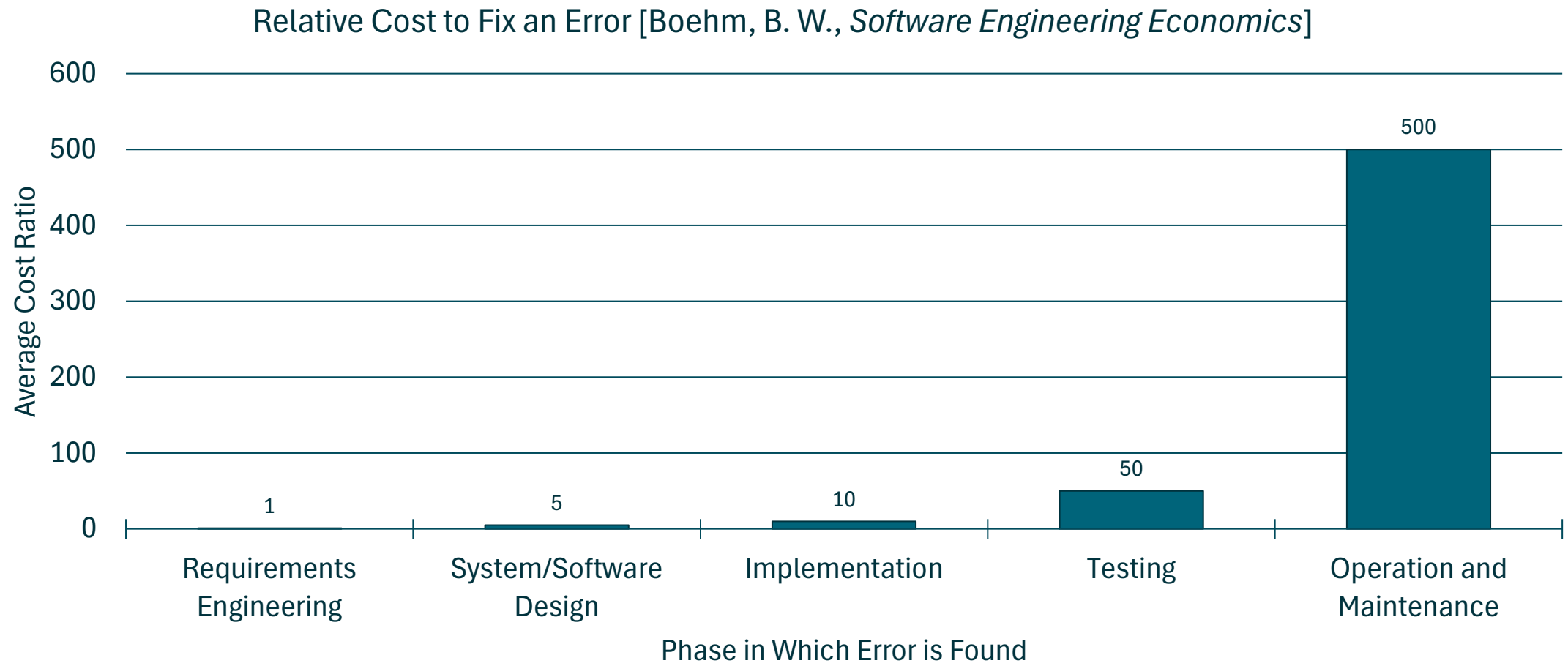
- 1.1. Course enrollment reports should be generated on the last working day of each month
- 1.2. A report should be generated for each course, displaying the total number of enrollments in the current month and the average age of the enrolled students
- 1.3. Access to the reports should be restricted to course managers
- 1.4. ...

Requirements: Different Levels

Such ambiguity is inevitable, as requirements may serve a dual function:

- **User Requirements** may be the basis for a bid for a contract
 - Customer may define general user requirements
 - Different contractors may propose different ways to meet the user requirements
- **System Requirements** may be the basis for the contract itself
 - Once a contract to develop the software has been awarded, the contractor formulates a more detailed set of System Requirements
 - So that the client understands exactly what the software will do, and can validate the proposal
 - Once accepted and validated, System Requirements can be put in the final contract and are binding!

Cost of Errors Through Project Lifecycle



Requirements Engineering (RE)

- RE is a sub-area of Software Engineering dealing with the process of defining the requirements for a software-to-be
- **Goal:**
 - Provide Software Engineers with methods, techniques and tools to understand and document what a software system must do

Functional and Non-functional Requirements

Requirements are often classified as functional or non-functional

- **Functional Requirements:**

- Services the system should provide
- How the system should react to particular inputs or behave in a given situation

- **Non-functional Requirements:**

- **Constraints** on the services or functions offered by the system
- Include timing constraints, process constraint, or standard-imposed constraints
- Often, apply to the whole system rather than individual features or services

Functional and Non-functional Requirements

In practice, the distinction between these two types of requirements is not cut-and-dried. Consider the following:

- *Only authorized users should be able to access the system*
- Seems like a non-functional requirement
- However, when developed in greater detail, it generates additional requirements that are clearly functional
 - E.g. *Users shall be able to perform the login and authenticate themselves*
- **Requirements are not independent and one requirement often generates or constrains other requirements**

Functional Requirements

- When expressed as **functional user requirements**, they may be written in natural language, so that they can be understood by non-technical people (e.g.: users and managers)
- When expressed as **functional system requirements**, they should be detailed, accurately describe system inputs and outputs and exceptions, so that software engineers know exactly what to implement

Non-functional Requirements

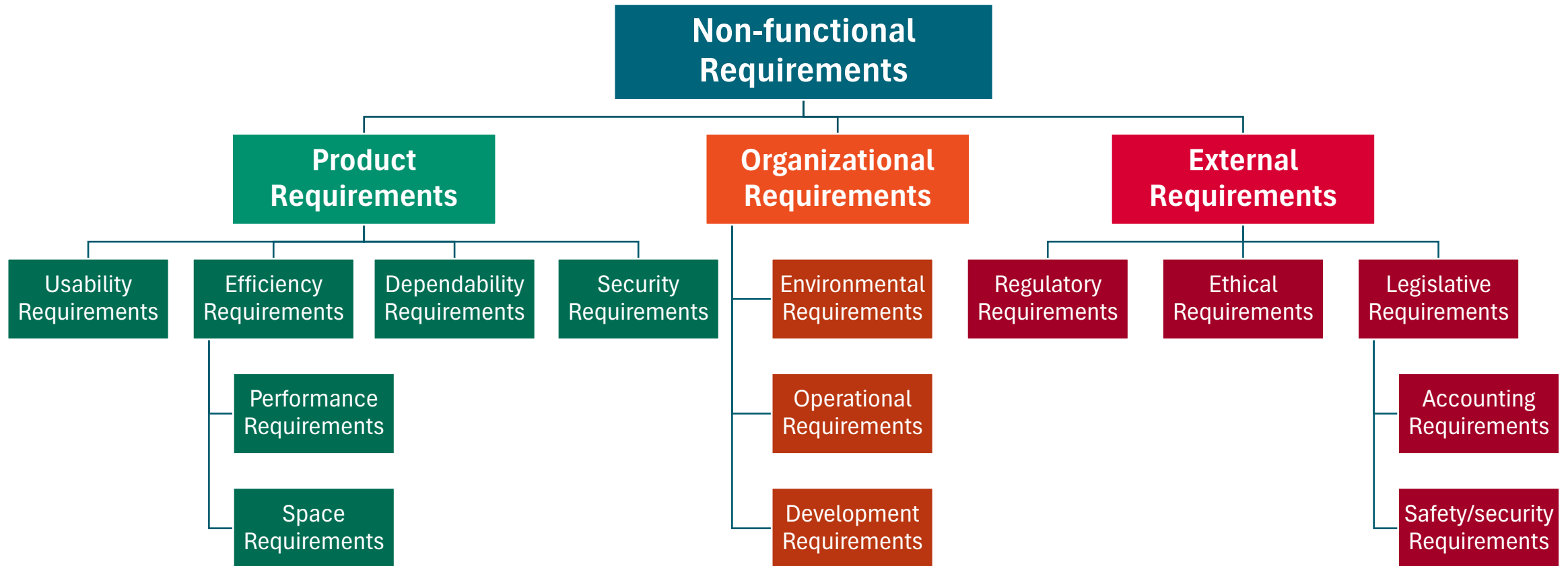
- Non-functional requirements are not directly concerned with specific services offered by the system
- Typically specify or constrain characteristics of the system as a whole
 - Constrains on how it should be implemented (e.g.: must use the Java language)
 - Specify other properties (e.g.: response times, memory use, ...)
- Often non-functional reqs are more critical than functional ones
 - Users may find a way around a sub-optimal implementation of a functional req..
 - ..but failing to meet a non-functional req may mean the system is unusable
 - A system that is not compliant with the GDPR cannot be deployed in Europe!

Types of Non-functional Requirements

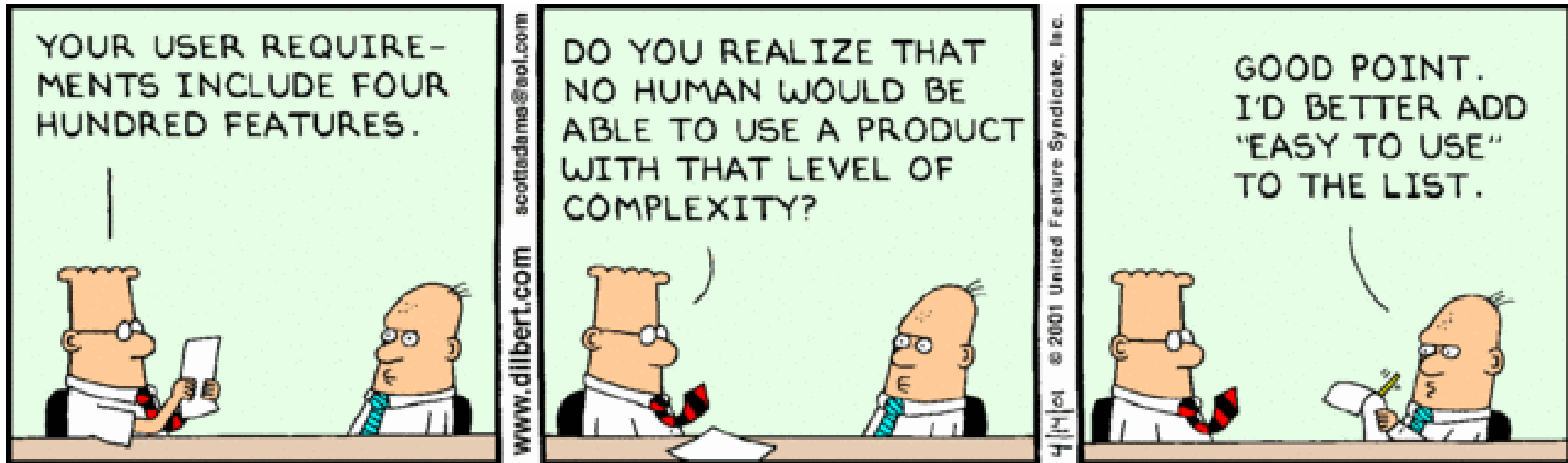
Non-functional requirements may arise from:

- Required characteristic of the product (**Product Reqs**)
 - How fast the system should be, how much memory it should require, usability requirements, acceptable failure rates, ...
- The customer's and the developers' organizations (**Organizational Reqs**)
 - Operational processes describing how the system will be used, required programming languages, requirements specifying the operational environment, ...
- External sources (**External Reqs**)
 - What must be done for the system to be approved by a third-party regulator, legal requirements, ethical requirements to ensure that the system will be acceptable to its users and the general public

Types of Non-functional Requirements



Dilbert on Usability Requirements



Domain Requirements

- **Domain requirements** are derived from the particular purpose or industry context in which a software is used
- Requirements for the control software of a medical device may include:
 - *The system safety shall be assured according to standard IEC 60601-1:Medical Electrical Equipment – Part 1: General Requirements for Basic Safety and Essential Performance.*
- The above requirement constrains both the design of the system and the development process.
- A perfectly functional software that was not developed according to the standard could not be used in practice!
- Domain requirements may be functional or non-functional.

Properties of Good Requirements

Requirements should be:

- **Clear and easy to understand** (especially User Requirements)
- **Unambiguous** (ambiguity leads to disputes with customers)
- **Complete**
- **Consistent** (i.e., should not conflict with each other)
- **Testable** (lack of testability leads to disputes with customers)
 - Given a requirement, it should be possible to **unambiguously determine** whether the system satisfies that requirement
- **Traceable**
 - Requirements have to be forward and backward linked to the other parts of the documentation, i.e. design, implementation and testing

The Demons of Ambiguity

Consider the traffic sign on the right

- To us, its intent is clear: keep kids safe in a school zone
- How would we explain it to a computer?
 - Why does the sign use the plural “*children*” instead of the singular “*child*”? Does the rule only apply if more than one child is present?
 - Who qualifies as a “child”? Is a 17-year-old a child?
 - What does it mean to be “present”?
 - Does this rule apply only when school is in session?



H. Robinson, The demons of ambiguity. [Full article archived here.](#)

The Demons of Ambiguity: Example

Requirement: The utility shall allow users to start/stop a service on a remote machine

- What if we start a service that's already running? Or try to stop a service that is not running?
- What if the remote machine reboots? Should the service restart?
- What if the connection drops?
 - Shall the utility display an error message?
 - Shall it try again?
 - After how much time? How many retries should be performed at most?

Requirements Testability

- It should be possible to **unambiguously determine** whether the system satisfies a requirement
- Otherwise, developers may argue that the requirement is satisfied, while the customer may not agree at all!
- System Functional Requirements should be as detailed as possible and leave no room to interpretation
- System Non-functional Requirements should include **quantitative** indicators whenever possible

Testable Non-functional Requirements

Not so testable requirements:

- a. The system shall be reliable
- b. The system shall be easy to use
- c. The system shall be fast and reactive

More testable requirements:

- a. The system shall have a monthly 99.9% uptime
- b. Users shall be able to use all system functions after at most two hours of training. After the training, the average number of user errors should not exceed two per hour of system use.
- c. The average response time of the system shall be no longer than 100ms

Metrics For Non-func. Requirements

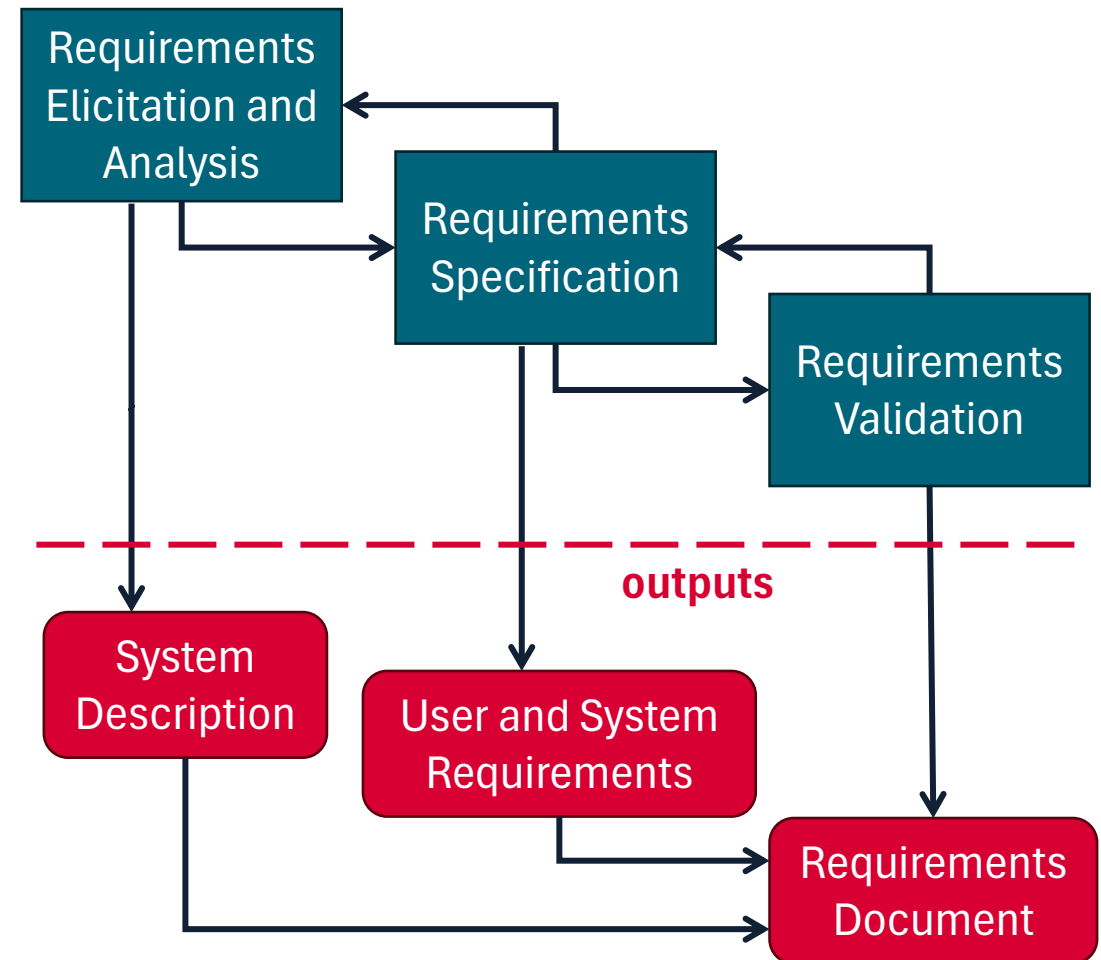
Property	Metrics
Performance	Processed operations / second User / event response time Screen refresh rate
Size	Megabytes
Ease of use	Required training time User error rate Number of requests for support
Reliability	Time to failure Availability rate (e.g.: uptime)
Robustness	Time to recover after failure Probability of data loss on failure

Requirements Engineering Processes

The Requirement Engineering Process

Three key steps:

- **Requirement Elicitation and Analysis:** Discover requirements by interacting with stakeholders
- **Requirement Specification:** Converting the requirements in a standardized form
- **Requirement Validation:** Checking that the requirements actually define the system the customer wants

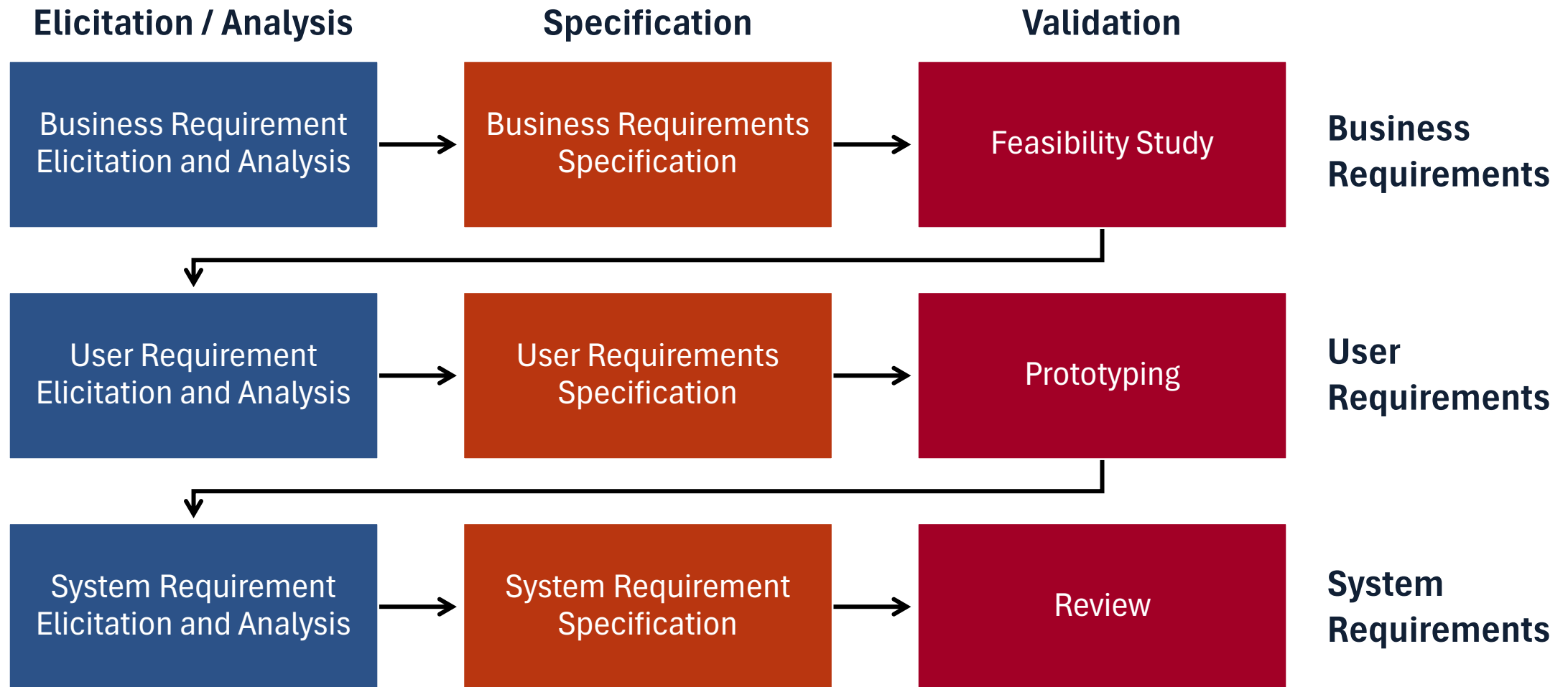


The Requirement Engineering Process

In practice, the RE process is not linear. Its activities are actually interleaved in an **iterative process** at different granularity levels

- In the first iteration, we focus on **Business-level Requirements**
 - Why is the project needed? Who benefits from it? When and where will it take place?
- Then, we define **User Requirements**
- Finally, we define **System Requirements**
- Each iteration includes elicitation and analysis, specification and validation phases

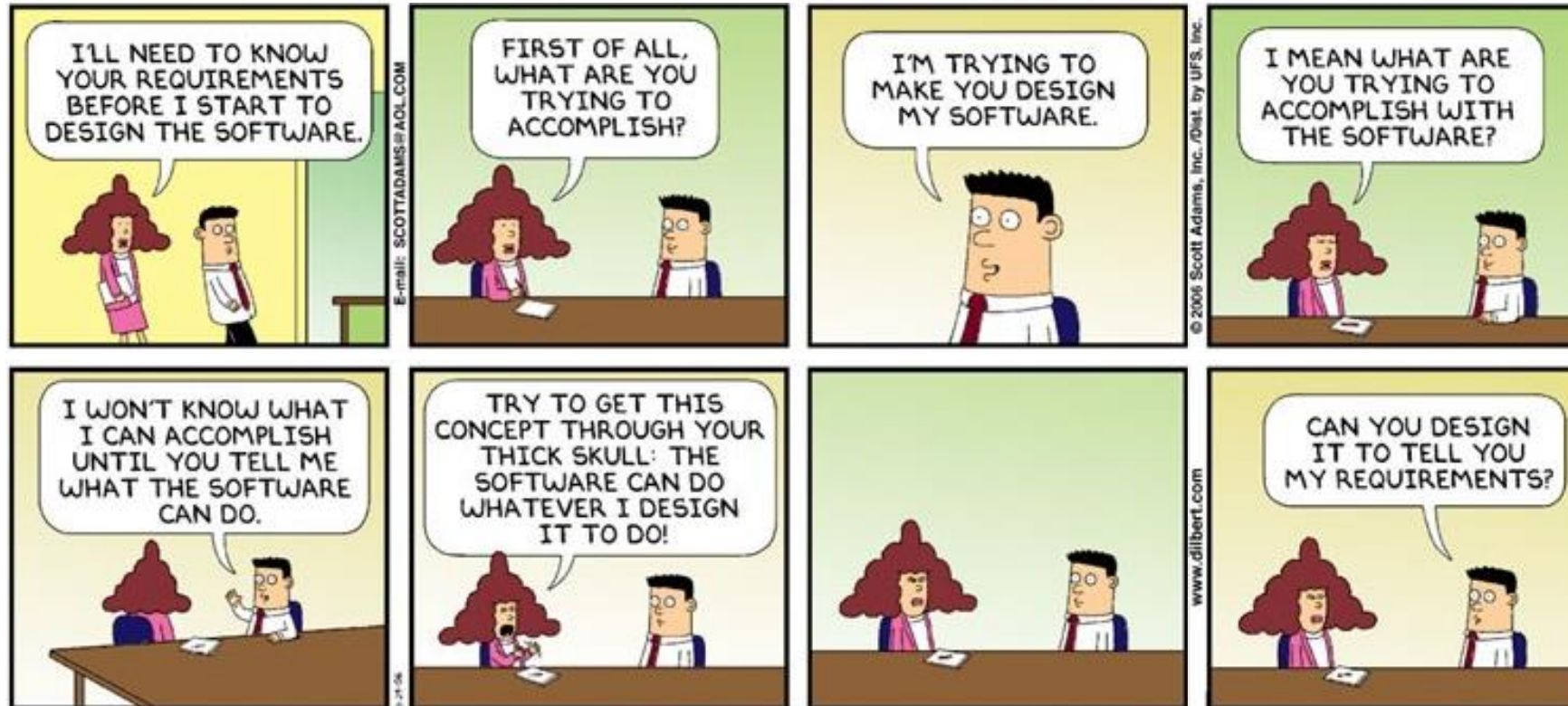
Iterative Requirement Engineering Process



Requirements Elicitation And Analysis

Requirement Elicitation and Analysis

- The goal of this activity is to find out what the customers want and need
- That's easier said than done!



Requirements Elicitation And Analysis

Requirement Elicitation is considered as the most critical part in RE

- Involves collaboration between different **stakeholders**
 - **Stakeholders** are the people or groups affected in some way by a software development project
 - End-users of the software. This may be an heterogeneous group. If the software-to-be is a order management system, users may be members of the sales department that put in the orders, members of the logistics departments that need to process the orders and deliver the items, or members of the customer care department.
 - People that use the outputs of the software
 - Client management and executives
 - Client technical staff, that may need to operate the system on a day-to-day basis

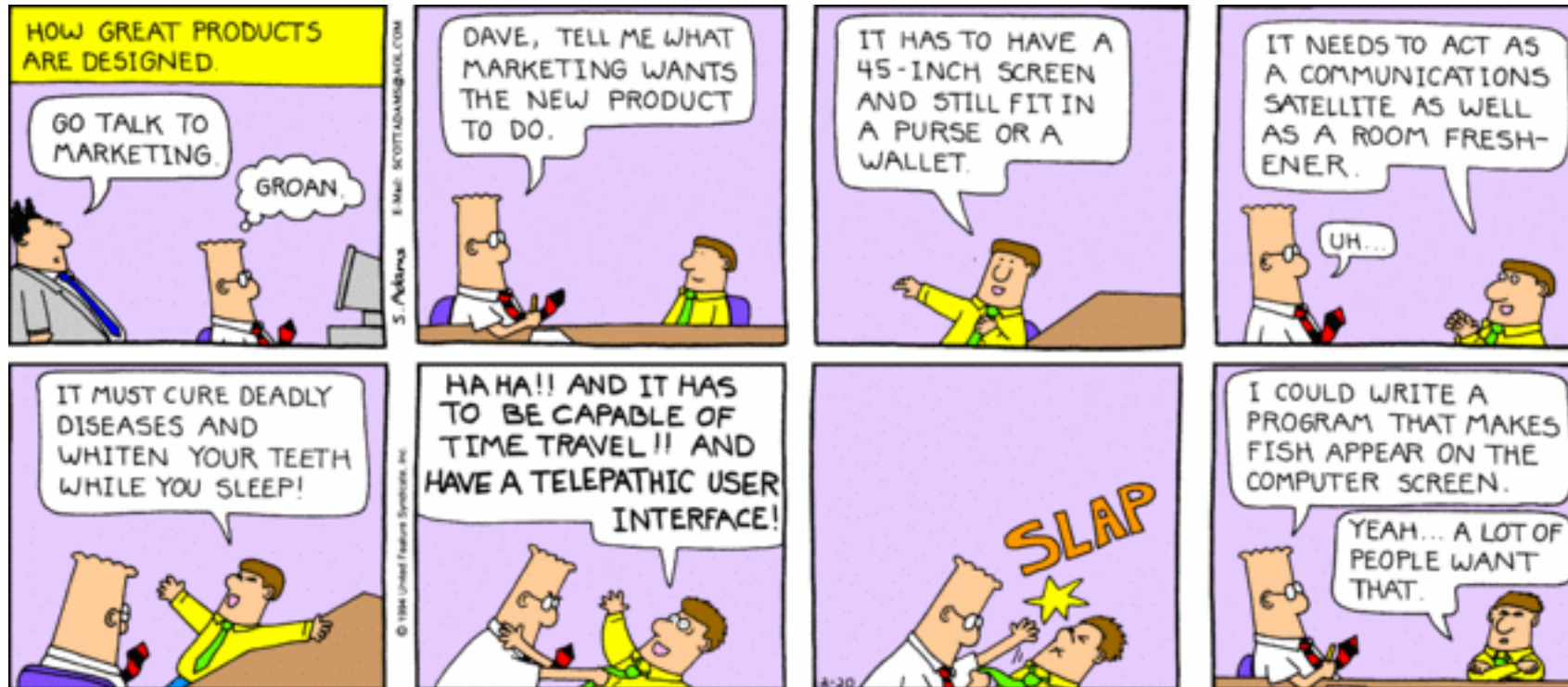
Requirement Elicitation: Challenges (1/2)

- Stakeholders **do not know what they want** from the software, except in the most general terms. They do not know what is feasible and what isn't, and **may make unrealistic demands**.
- Stakeholders are experts in their own domain. They naturally express requirements in their own terms and jargon, with implicit knowledge of their work. They may give some details for granted, when they're not.
- Different stakeholders, with different requirements, may express their requirements in different ways. Requirement Engineers need to work their way around **commonalities** and **conflicts**

Requirement Elicitation: Challenges (2/2)

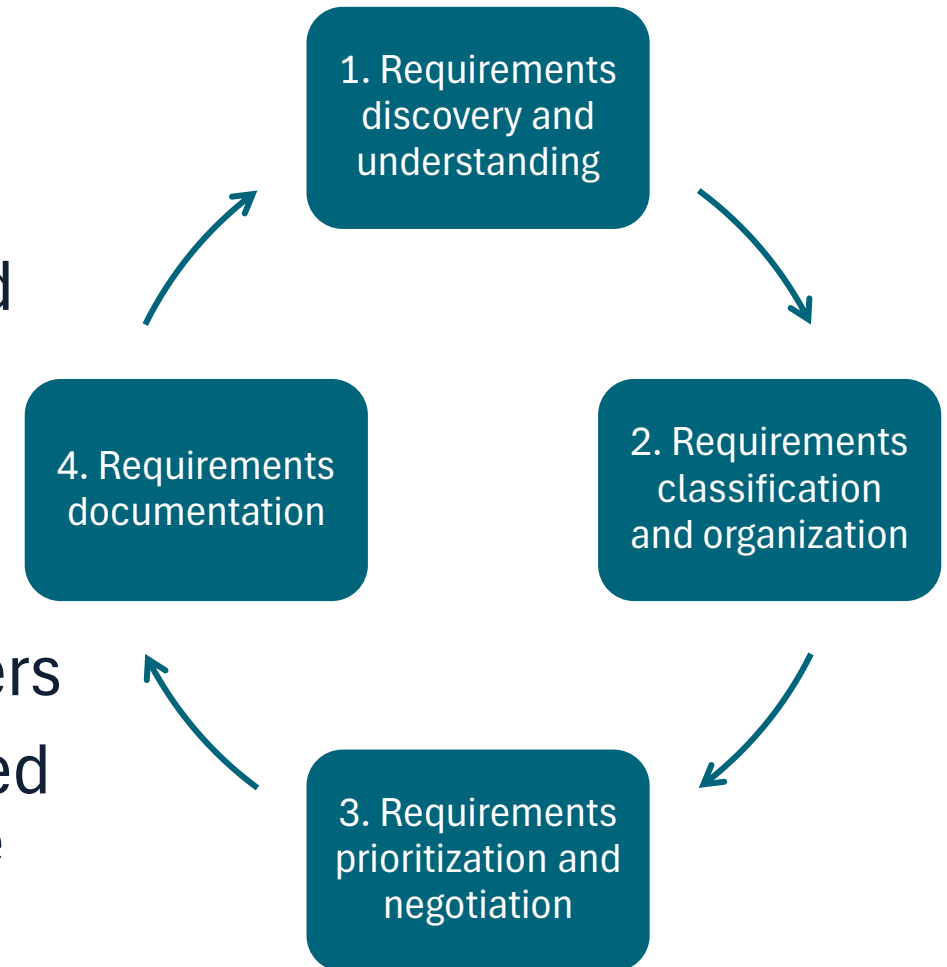
- Political factors may influence the elicitation process. Managers may demand specific requirements because these would allow them to increase their influence in the organization.
- Stakeholders have different (and sometimes **conflicting**) views on the importance and priority of requirements. If some stakeholders feel that their views have not been properly considered, then they may deliberately attempt to undermine the RE process.
- Requirements **will change** during the elicitation process. New requirements may emerge from new stakeholders that were not initially considered.

Dilbert on RE Challenges



Requirement Elicitation And Analysis Process

- **Discovery and understanding:** interact with stakeholders to discover requirements
- **Classification and organization:** related requirements are grouped together and organized
- **Prioritization and negotiation:** solve requirements conflicts arising from conflicting needs of different stakeholders
- **Documentation:** keep track of discovered requirements for the next iteration of the elicitation process



Requirements Elicitation Techniques

Different approaches can be used and combined to discover requirements

- Interviews with stakeholders
- Observation and Ethnographic Studies
- Definition of Personas
- Definition of Scenarios and User Stories
- Definition of mock-ups and prototypes

Interviews with Stakeholders

Formal or informal interviews with stakeholders are fundamental in RE

- During interviews, the RE team asks the stakeholders about their work, the system they currently use, and the system-to-be-developed
- Requirements are derived from the answers to these questions
- Interviews may be:
 - **Closed interviews:** stakeholders answer a pre-defined list of questions
 - **Open interviews:** there is no pre-defined agenda
 - In practice, interviews are usually a combination of both approaches. Completely open-ended interviews rarely work well. It's better to start from a set of questions defined in advance, which usually lead to other issues that are discussed in a less structured way.

Interviews with Stakeholders

- People generally like talking about their job, and take part in interviews with a positive attitude
- However, stakeholders often have their own job to do, you should not waste their time with useless meetings, and should try to minimize the impact of interviews on their job
- Challenges:
 - Stakeholders use jargon and give some requirements for granted
 - Stakeholders know about their job, they may have a partial or distorted vision of the job of other colleagues in different areas or departments
 - Stakeholders may be reluctant to discuss organizational requirements and constraints due to subtle power dynamics in the organization

Real life story time

- Back in my days, I did my curricular internship for the B.Sc. in Computer Science at the **Central Direction for Financial Services** at the **Municipality of Naples**
- Developed a full-stack web app to support the management of a minor tribute (approx. tax revenue: € 1-2 M per year). Before, the tribute was managed mostly via Excel spreadsheets.
- Project started from scratch, so – being a good SE student – I started with Requirement Elicitation and interviewed some stakeholders



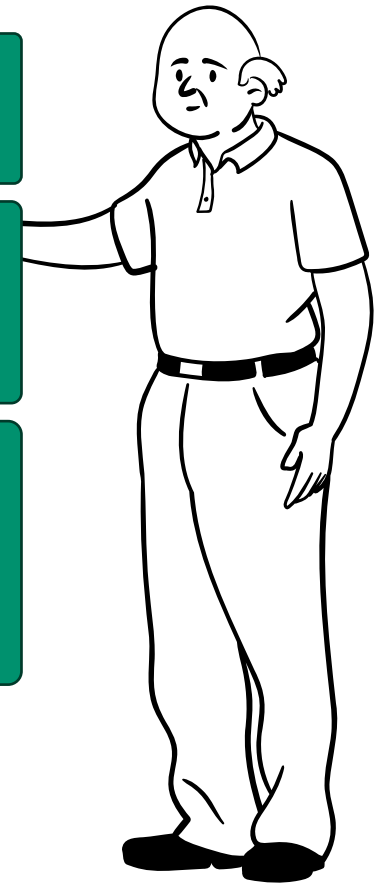
Real life story time

Signor Catello, la ringrazio innanzitutto per la sua disponibilità. Mi racconta come funziona, dal suo punto di vista, la gestione di questo tributo?

Certo! È molto semplice! Innanzitutto, un operatore sul campo mi invia la lettura per un dato anno per un certo contribuente.

Quindi, inserisco la lettura in una maschera che calcola l'imponibile. Successivamente, faccio la stampa delle ingiunzioni di pagamento, che vengono poi emesse.

Una volta notificata l'ingiunzione, il contribuente ha 60 giorni per pagare. Se non paga entro il termine, devo procedere all'iscrizione al ruolo coattivo. Per l'iscrizione, inserisco i dati in una maschera che genera il Tracciato 290 da trasmettere all'ente di riscossione.



Ethnography

- Software rarely exist in isolation, and is used in a social and organizational environment
 - Requirements can be generated or constrained by such environment
 - Often, the organization and processes that are **actually** used in an organization are different from the formal processes that are supposed to be used
- **Ethnography** is an observational technique that can be used to understand operational processes
 - A requirement engineer immerses themselves in the working environment of end users, where the system will be used
 - Day-to-day work is observed, taking notes of the actual tasks and participants involved

Ethnography

Ethnography is particularly effective for discovering:

- Requirements derived from the way in which people actually work, rather than the way in which business process definitions say they ought to work.
- Requirements derived from cooperation and awareness of other people's activities.
 - E.g.: Before sending a customer feedback survey, a Customer Care employee checks with his colleagues in the logistics department that the order has actually been delivered to that customer.
 - “Customer care employees shall be able to check whether a customer has already received an order”

Stories

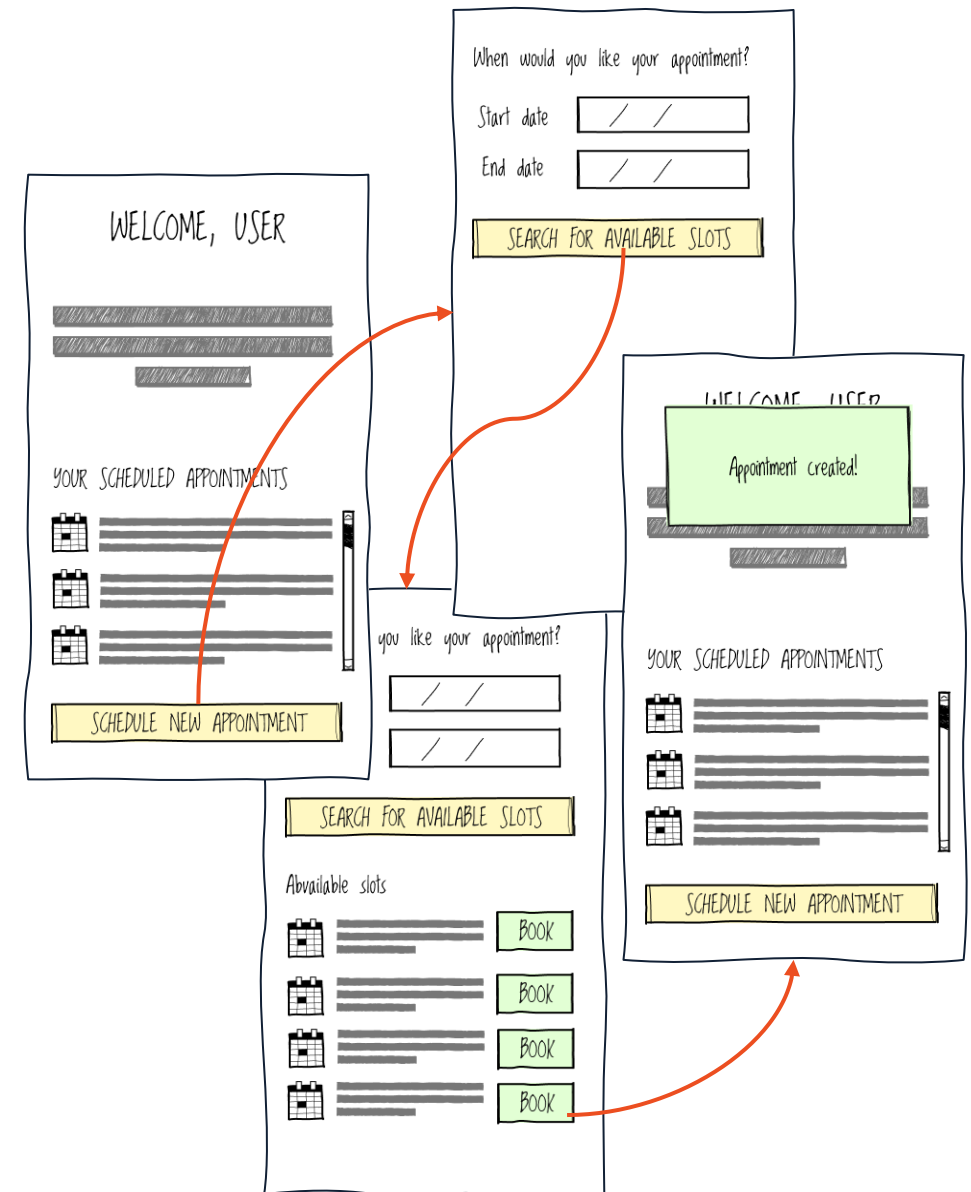
- Stories are written as narrative text and present an high-level description of the system use
- They are effective in presenting relatable “big picture” goals

Story: Scheduling an appointment by using the app

Pam needs to schedule a dental care appointment because her wisdom tooth is starting to hurt. She logs into the system and searches for available slots in the same day or tomorrow. The system shows a list of available slots. Pam selects one of the proposed slots and confirms the booking. Pams receives a confirmation message via email, with the details of the booked appointment.

Low-fidelity mockups

- Low-fidelity mockups (or **wireframes**) are **simplified sketches** of the System user interfaces
- Quick and cost-effective way to visualize the structure and basic flows of the system
- Focus is functionality and flow, not aesthetics!



Low-fidelity mockups: benefits

- **Facilitate Understanding:** Help stakeholders grasp the system's basic functionality. Serve as a starting point for discussions, ensuring clear communication of initial ideas.
- **Help discovering new requirements:** it is easier to reason about concrete interfaces rather than abstract statements.
- **Engagement and Iterative Feedback:** Promotes stakeholder engagement and allows for rapid iteration and feedback without significant investment in design.
- **Foundation for Development:** Provides a solid foundation for moving towards higher-fidelity designs and eventual development.

Low-fidelity mockups: tools

Paper and Pencil:

- Quick and accessible for initial sketches.
May get messy for complex systems.



Digital Tools:

- Commercial software like [Balsamiq](#) or [Figma](#) allows for creating and sharing wireframes electronically
- Open-source alternatives such as [mydraft.cc](#), [draw.io](#) or [Pencil Project](#) also exist





How the customer explained it



How the Project Leader understood it



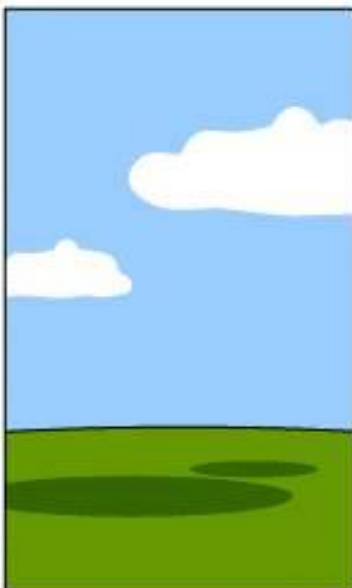
How the Analyst designed it



How the Programmer wrote it



How the Business Consultant described it



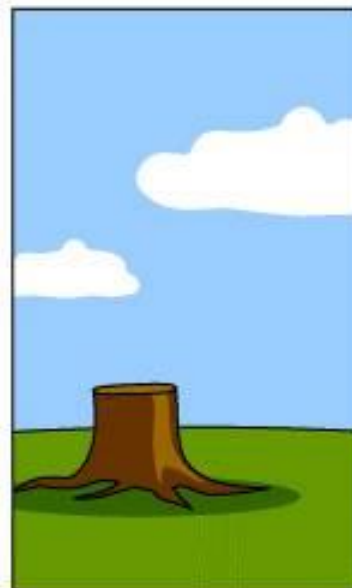
How the project was documented



What operations installed



How the customer was billed



How it was supported



What the customer really needed

Requirements Specification

Requirement Specification

- The process of writing down the user and system requirements in a **requirements document**.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
 - It is therefore important that these are as complete and detailed as possible.

Requirements and Design

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, **requirements** and **design** are **inseparable**
 - Requirements may be structured and organized based on a high-level system architecture
 - The system may inter-operate with other systems that generate design requirements
 - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement

Requirements Specification

Different approaches are possible:

- **Natural Language:** express requirements as numbered sentences in natural language. Each sentence should express one requirement.
- **Structured Natural Language:** Use a standardized form or template
- **Semi-formal notations and models:** UML Use Case Diagrams and other domain models, typically supplemented by natural language annotations
- **Formal Specification:** These notations are based on mathematical concepts such as finite and infinite state machines, temporal logics

Requirements Specifications

Different approaches are used in different domains

- When engineering **safety-critical** systems, it is common to use formal specifications and structured natural language
- When engineering a to-do list app, one might use unstructured natural language to express requirements
- When engineering a medium-to-large sized information system, leveraging semi-formal notations and models might be a good compromise

Natural Language (NL) Specifications

- Requirements are written as natural language sentences, optionally supplemented by diagrams and tables.
- Used for writing requirements because it is **expressive, intuitive** and **universal**.
 - This means that the requirements can be understood by users and customers.
- Can express **functional** and **non-functional** requirements

Guidelines for NL Specification

- Define a “standard” format and use it for all requirements.
 - E.g.: <part of the system> shall <requirement> (<rationale>)
 - E.g.: The overall monthly enrollment report shall include statistics about the gender of enrolled students (this is required to fill the gender equality report)
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

Problems with Natural Language

- **Lack of clarity**
 - Precision is difficult without making the document difficult to read.
- **Requirements confusion**
 - Functional and non-functional requirements tend to be mixed-up.
- **Requirements amalgamation**
 - Several different requirements may be expressed together.

Structured Specifications

- An approach to writing requirements where the freedom of the requirements writer is limited and a standardized way of writing requirements is enforced.
- This works well for some types of requirements (e.g.: requirements for embedded control systems) but is sometimes too rigid for writing business system requirements.

Structured Specification Example

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level between 3 and 7 units.
Inputs	Current sugar reading (r_2), the previous two readings (r_0 and r_1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose – the dose in insulin to be delivered.
Action	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requires	Two previous readings, so the rate of change of sugar level can be computed
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	In the memory, r_0 is replaced by r_1 , then r_1 is replaced by r_2
Side effects	None

IEEE standard 830 -1998

- It defines a generic structure for a requirements document that must be instantiated for each specific system. It contains several chapters:
 - Introduction.
 - General description.
 - Specific requirements.
 - Appendices.
 - Index.

IEEE standard 830 -1998

Table of contents

1.Introduction

- 1.1. Purpose (of the document)
- 1.2. Scope (of the product)
- 1.3. Definitions, Acronyms and Abbreviations
- 1.4. References
- 1.5. Overview (General description of the organization of the document)

2.General Description

- 2.1. Product Perspective (in which context will the product be used ? / will it interact with existing systems?)
- 2.2. Product Functions
- 2.3. User Characteristics
- 2.4. General Constraints (e.g. laws or rules to be respected)
- 2.5. Assumptions and Dependencies (e.g. existing software or hardware)

....

IEEE standard 830 -1998

3. Specific Requirements

3.1. Functional Requirements

3.1.1. Functional Requirement 1

3.1.1.1. Introduction

3.1.1.2. Inputs

3.1.1.3. Processing

3.1.1.4. Output

3.1.2. Functional Requirement 2

...

3.1.3. Functional Requirement n

...

IEEE standard 830 -1998

3.2. External Interface Requirements

- 3.2.1. User Interfaces
- 3.2.2. Hardware Interfaces
- 3.2.3. Software Interfaces
- 3.2.4. Communications Interfaces

3.3. Performances Requirements

3.4. Design Constraints

- 3.4.1 Standard Compliance
- 3.4.2. Hardware Limitation

...

3.5. Software System Attributes

- 3.5.1. Security
- 3.5.2. Maintainability

...

3.6. Other Requirements

- 3.6.1. Logical Database Requirements (requirements related to data management)
- 3.6.2. Operations (requirements related to how the system is operated – e.g. authentication requirements)
- 3.6.3. Site Adaptation requirements

...

Update : ISO/IEC/IEEE 29148:2018

- A new ISO standard have been released in 2011 and updated in 2018
 - Systems and software engineering — Life cycle processes requirements engineering

Writing requirements

- Requirements are mandatory binding provisions and use 'shall'.
- Non-requirements, such as descriptive text, use verbs such as 'are', 'is', and 'was'. It is best to avoid using the term 'must', due to potential misinterpretation as a requirement.
- Statements of fact, futurity, or a declaration of purpose are non-mandatory, non-binding provisions and use 'will'. 'Will' can also be used to establish context or limitations of use.
- Preferences or goals are desired, non-mandatory, non-binding provisions and use 'should'. They are not requirements.
- Suggestions or allowances are non-mandatory, non-binding provisions and use 'may'.
- Use positive statements and avoid negative requirements such as 'shall not'.
- Use active voice: avoid using passive voice, such as 'it is required that'.
- Avoid using terms such as 'shall be able to'.

Examples of Functional requirements

[Condition] [Subject] [Action] [Object] [Constraint of Action]

EXAMPLE: When signal x is received **[Condition]**, the system **[Subject]** shall set **[Action]** the signal x received bit **[Object]** within 2 seconds **[Constraint of Action]**.

Or

[Condition] [Subject] [Action] [Object] [Constraint of Action]

EXAMPLE: At sea state 1 **[Condition]**, the Radar System **[Subject]** shall detect **[Action]** targets **[Object]** at ranges out to 100 nautical miles **[Constraint of Action]**.

Or

[Subject] [Action] [Constraint of Action]

EXAMPLE: The Invoice System **[Subject]** shall display pending customer invoices **[Action]** in ascending order of invoice due date **[Constraint of Action]**.

Writing requirements

Vague and general terms shall be avoided. They result in requirements that are often difficult or even impossible to verify or may allow for multiple interpretations. The following are types of unbounded or ambiguous terms:

- superlatives (such as 'best', 'most');
- subjective language (such as 'user friendly', 'easy to use', 'cost effective');
- vague pronouns (such as 'it', 'this', 'that');
- ambiguous terms such as adverbs and adjectives (such as 'almost always', 'significant', 'minimal') and ambiguous logical statements (such as 'or', 'and/or');

NOTE 1 Consider multiple requirements when encountering terms such as 'or', 'and', or 'and/or'.

- open-ended, non-verifiable terms (such as 'provide support', 'but not limited to', 'as a minimum');
- comparative phrases (such as 'better than', 'higher quality');
- loopholes (such as 'if possible', 'as appropriate', 'as applicable');
- terms that imply totality (such as 'all', 'always', 'never', and 'every');

NOTE 2 It is very difficult to verify such requirements.

- incomplete references (not specifying the reference with its date and version number; not specifying just the applicable parts of the reference to restrict verification work).

Update : ISO/IEC/IEEE 29148:2018

Business Requirements Specification

”describes the organization’s motivation for why the system is being developed or changed, defines processes and policies/rules under which the system is used and documents the top-level requirements from the stakeholder perspective including expressing needs of users/operators/maintainers as derived from the context of use in a specific, precise and unambiguous manner.

The BRS serves as the basis of the stakeholders' active participation in the requirement processes.”

1. Introduction

- 1.1 Business purpose
- 1.2 Business scope
- 1.3 Business overview
- 1.4 Definitions
- 1.5 Major stakeholders

2. References

3. Business management requirements

- 3.1 Business environment
- 3.2 Mission, goals, and objectives
- 3.3 Business model
- 3.4 Information environment

4. Business operational requirements

- 4.1 Business processes
- 4.2 Business operational policies and rules
- 4.3 Business operational constraints
- 4.4 Business operational modes
- 4.5 Business operational quality
- 4.6 Business structure

5. Preliminary operational concept of proposed system

- 5.1 Preliminary operational concept
- 5.2 Preliminary operational scenarios

6. Other preliminary life-cycle concepts

- 6.1 Preliminary acquisition concept
- 6.2 Preliminary deployment concept
- 6.3 Preliminary support concept
- 6.4 Preliminary retirement concept

7 Project Constraints

8. Appendix

- 8.1 Acronyms and abbreviations

Update : ISO/IEC/IEEE 29148:2018

Stakeholder Requirement Specification

”The Stakeholder Requirements Specification (StRS) describes the organization’s motivation for why the system is being developed or changed, defines processes and policies/rules under which the system is used and documents the top-level requirements from the stakeholders’ perspective including expressing needs of users/operators/maintainers as derived from the context of use in a specific, precise and unambiguous manner.

In the context described in the BRS, the StRS describes how the organization will utilize the system as a means to contribute to the business. ”

1. Introduction

- 1.1 Stakeholder purpose
- 1.2 Stakeholder scope
- 1.3 Overview
- 1.4 Definitions
- 1.5 Stakeholders

2. References

3. Business management requirements

- 3.1 Business environment
- 3.2 Mission, goals, and objective
- 3.3 Business model
- 3.4 Information environment

4. System operational requirements

- 4.1 System processes
- 4.2 System operational policies and rules
- 4.3 System operational constraints
- 4.4 System operational modes and states

5. User requirements

6. Detailed Life-cycle concepts of proposed system

- 6.1 Operational concept
- 6.2 Operational scenarios
- 6.3 Acquisition concept
- 6.4 Deployment concept
- 6.5 Support concept
- 6.6 Retirement concept

7 Project Constraints

8. Appendix

- 8.1 Acronyms and abbreviations

ISO/IEC/IEEE 29148:2018

System Requirement Specification

” The System Requirements Specification (SyRS) identifies the technical requirements for the selected system-of-interest and usability for the envisaged human-system interaction. It defines the high-level system requirements from the domain perspective, along with background information about the overall objectives for the system, its target environment and a statement of the constraints, assumptions and non-functional requirements. It may include conceptual models designed to illustrate the system context, usage scenarios, the principal domain entities, data, information and workflows.

The purpose of the SyRS is to provide a description of what the system should do, in terms of the system's interactions or interfaces with its external environment. ”

1. Introduction

- 1.1 System purpose
- 1.2 System scope
- 1.3 System overview
 - 1.3.1 System context
 - 1.3.2 System functions
 - 1.3.3 User characteristics
- 1.4 Definitions

2. References

3. System requirements

- 3.1 Functional requirements
- 3.2 Usability requirements
- 3.3 Performance requirements
- 3.4 Interface requirements
 - 3.4.1 External interface requirements
 - 3.4.2 Internal interface requirements
- 3.5 System operations
- 3.6 System modes and states
- 3.7 Physical characteristics
- 3.8 Environmental conditions
- 3.9 Security requirements
- 3.10 Information management requirements
- 3.11 Policy and regulation requirements
- 3.12 System life cycle sustainment requirements
- 3.13 Packaging, handling, shipping and transportation requirements

4. Verification

(parallel to subsections in Section 3)

5. Appendices

- 5.1 Assumptions and dependencies
- 5.2 Acronyms and abbreviations

ISO/IEC/IEEE 29148:2018

Software Requirement Specification

” The software requirements specification (SRS) is a specification for a particular software product, program, or set of programs that performs certain functions in a specific environment. The SRS may be written by one or more representatives of the supplier, one or more representatives of the acquirer, or by both. ”

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Product overview
 - 1.3.1 Product perspective
 - 1.3.2 Product functions
 - 1.3.3 User characteristics
 - 1.3.4 Limitations
- 1.4 Definitions

2. References

3. Requirements

- 3.1 Functions
- 3.2 Performance requirements
- 3.3 Usability requirements
- 3.4 Interface requirements
- 3.5 Logical database requirements
- 3.6 Design constraints
- 3.7 Software system attributes
- 3.8 Supporting information

4. Verification

(parallel to subsections in Section 3)

5. Appendices

- 5.1 Assumptions and dependencies
- 5.2 Acronyms and abbreviations

ISO/IEC/IEEE 29148:2018

- More in details : **Functions**
 - Define the fundamental actions that have to take place in the software in accepting and processing the inputs and in processing and generating the outputs, including:
 - a) validity checks on the inputs;
 - b) exact sequence of operations;
 - c) responses to abnormal situations, including:
 - 1) overflow;
 - 2) communication facilities;
 - 3) hardware faults and failures; and
 - 4) error handling and recovery;
 - d) effect of parameters;
 - e) relationship of outputs to inputs, including:
 - 1) input/output sequences; and
 - 2) formulas for input to output conversion.